

Guion Detallado para Video de 4 Minutos

Archivo local de apoyo. No subir a GitHub.

Objetivo del video

Mostrar que el proyecto funciona de extremo a extremo, explicar cómo usaste IA durante el desarrollo y demostrar que entiendes el código y las decisiones técnicas. La duración máxima es 4 minutos.

Preparación antes de grabar

Antes de iniciar Loom:

1. Tener Docker corriendo.
2. Tener `.env` con `GROQ_API_KEY` real.
3. Levantar el proyecto:

```
docker compose up --build
```

4. Tener abierto:
 - terminal con Docker Compose
 - `docs/requests.http` o Postman/Thunder Client
 - `src/services/lead.service.js`
 - `src/services/groq.service.js`
 - `src/validators/lead.validator.js`
 - `README.md`

Si no quieres mostrar la API key, no abras `.env` en cámara.

Estructura recomendada

1. Demo funcional (~1 minuto)

Qué mostrar:

- `GET /health`
- `POST /leads`
- `GET /leads`

Guion sugerido:

"Primero voy a mostrar el flujo funcionando. Este proyecto es un microservicio sin frontend hecho con Node.js, Express, PostgreSQL y Groq. Está levantado con Docker Compose, con un contenedor para la API y otro para PostgreSQL."

Mostrar:

```
GET http://localhost:3000/health
```

Decir:

"El healthcheck responde `status ok`, así confirmo que la API está arriba."

Luego mostrar:

```
POST http://localhost:3000/leads
Content-Type: application/json

{
  "nombre_empresa": "Acme",
  "dominio": "acme.com",
  "cargo_contacto": "Gerente Comercial"
}
```

Decir:

"Ahora creo un lead con nombre de empresa, dominio y cargo del contacto. El servicio valida el input, guarda la empresa, llama a Groq para generar un correo breve de outreach y guarda ese resultado en la base."

Mostrar respuesta `201` con:

- company
- outreach
- correo_generado
- llm_provider
- llm_model
- attempts
- status

Luego mostrar:

```
GET http://localhost:3000/leads
```

Decir:

"Finalmente listo los leads generados. Acá se ve el correo guardado, el modelo usado, la cantidad de intentos y el estado del lead."

2. Flujo con Claude Code o IA (~1,5 minutos)

Qué mostrar:

Puedes mostrar una parte del chat donde pediste:

- revisar estructura antes de modificar
- implementar el plan
- agregar retries
- mitigar prompt injection
- mejorar README
- agregar estados del lead

Guion sugerido:

"Durante el desarrollo usé Claude Code como apoyo, pero no le pedí simplemente que escribiera todo sin revisar. Primero le di contexto del stack, la estructura de carpetas y los requisitos de la prueba. Antes de implementar, le pedí que revisara la estructura existente para detectar archivos vacíos o problemas."

Mostrar una parte donde se vea que se revisaron archivos.

"Después trabajé con un plan concreto: separar responsabilidades entre routes, controller, service y db; usar SQL parametrizado; manejar errores de forma centralizada; y agregar reintentos asíncronos para Groq."

Mostrar alguna respuesta o diff.

"También fui revisando lo que generaba. Por ejemplo, ajusté el README para que fuera más claro, pedí agregar estados simples del lead y reforcé el prompt cuando vi que el modelo podía devolver placeholders o frases demasiado genéricas. La IA me ayudó a acelerar, pero yo fui guiando las decisiones y validando que el código cumpliera los requisitos."

Punto importante:

"No dejé credenciales hardcodeadas; todo va por variables de entorno."

3. Código explicado en tus palabras (~1 minuto)

Archivo recomendado para explicar: `src/services/lead.service.js`

Es el mejor porque muestra el flujo completo.

Guion sugerido:

"La parte que quiero explicar es `lead.service.js`, porque concentra la lógica principal sin mezclarla con Express ni con SQL directo."

Explicar:

1. `upsertCompany(leadData)`

"Primero guardo o actualizo la empresa. El estado inicial queda como `pending`, porque todavía no sé si el correo se va a generar correctamente."

2. `retryAsync`

"Después llamo a Groq dentro de `retryAsync`. Esto permite hacer hasta 3 intentos asíncronos con backoff: el primero inmediato, luego espera 500 milisegundos y finalmente cerca de 1500 milisegundos. Esto cubre fallos temporales del LLM, pero no pretende resolver escalabilidad masiva."

3. éxito

"Si Groq responde bien, guardo el correo en `lead_outreach` junto con metadata como provider, modelo, status y cantidad de intentos. Luego actualizo la empresa a `processed`."

4. error

"Si los 3 intentos fallan, guardo el error en `lead_errors`, marco la empresa como `error` y lanzo un `ApiError` con status 502. El cliente recibe un mensaje controlado, no un stack trace."

Cerrar:

"Elegí esta separación porque hace el código fácil de explicar: el controller maneja HTTP, el service maneja negocio, las queries manejan SQL y el middleware maneja errores."

Si decides mostrar `src/services/groq.service.js`:

"En el prompt aclaro que los datos del lead son externos y no confiables. Además le pido al modelo que responda JSON con un campo `email`; así puedo parsear una salida estructurada. Luego valido ese email: si viene vacío, trae placeholders o claims no soportados, lo trato como error reintentable."

Cierre recomendado (~15 segundos)

"En resumen, el servicio funciona de extremo a extremo: valida input, protege el prompt contra casos básicos de prompt injection, genera correos con Groq, guarda resultados en PostgreSQL y registra errores si el LLM falla. Para producción agregaría cola, workers, rate limiting, observabilidad y manejo seguro de secretos."

Checklist para no olvidar durante la grabación

- Mostrar `docker compose ps` o terminal con contenedores arriba.
- Mostrar `GET /health`.
- Crear un lead real.
- Listar leads.
- Mostrar una interacción real con Claude Code.
- Explicar `lead.service.js`.
- Mencionar retries y errores.
- Mencionar prompt injection.
- Mencionar que el prompt evita placeholders y afirmaciones inventadas.
- Mencionar estados `pending`, `processed`, `error`.
- No mostrar API keys.

Frases cortas útiles

- "Separé responsabilidades para que el proyecto sea fácil de mantener."
- "Uso SQL parametrizado para no concatenar input de usuario."

- "Los datos del lead se tratan como información externa no confiable."
- "Los retries son para resiliencia, no para procesar carga masiva."
- "Si Groq falla, el sistema registra el error y responde de forma controlada."
- "En producción desacoplaría esto con una cola y workers."